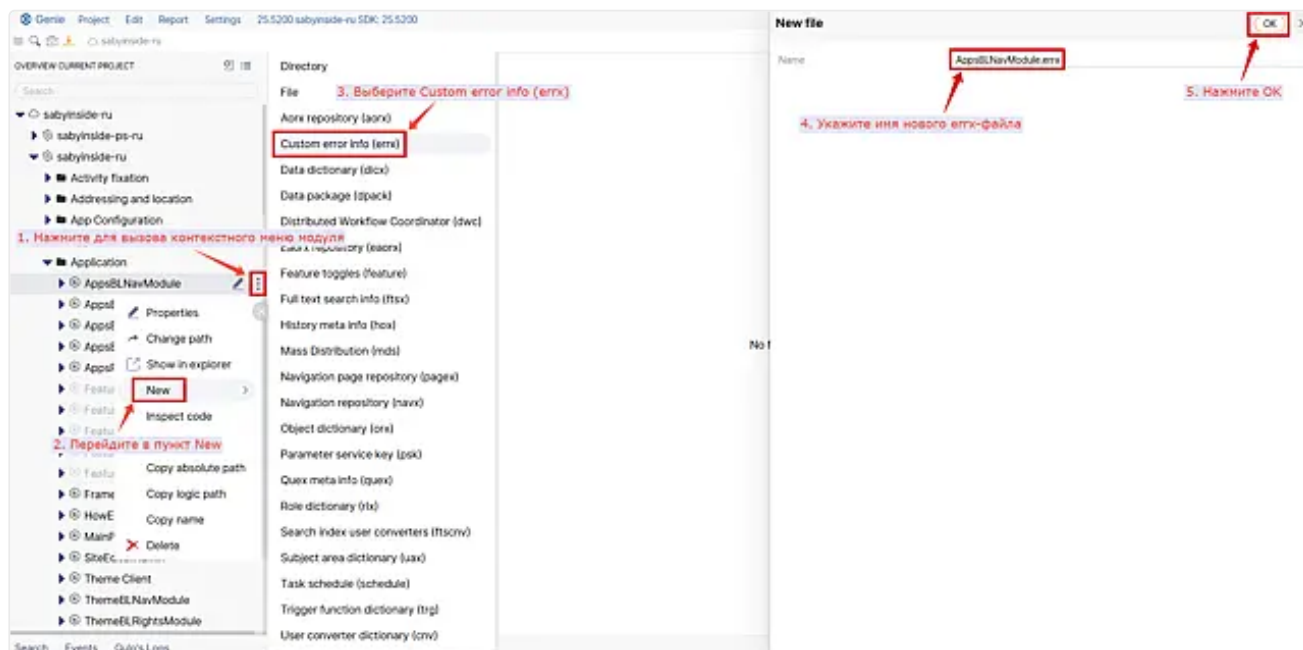


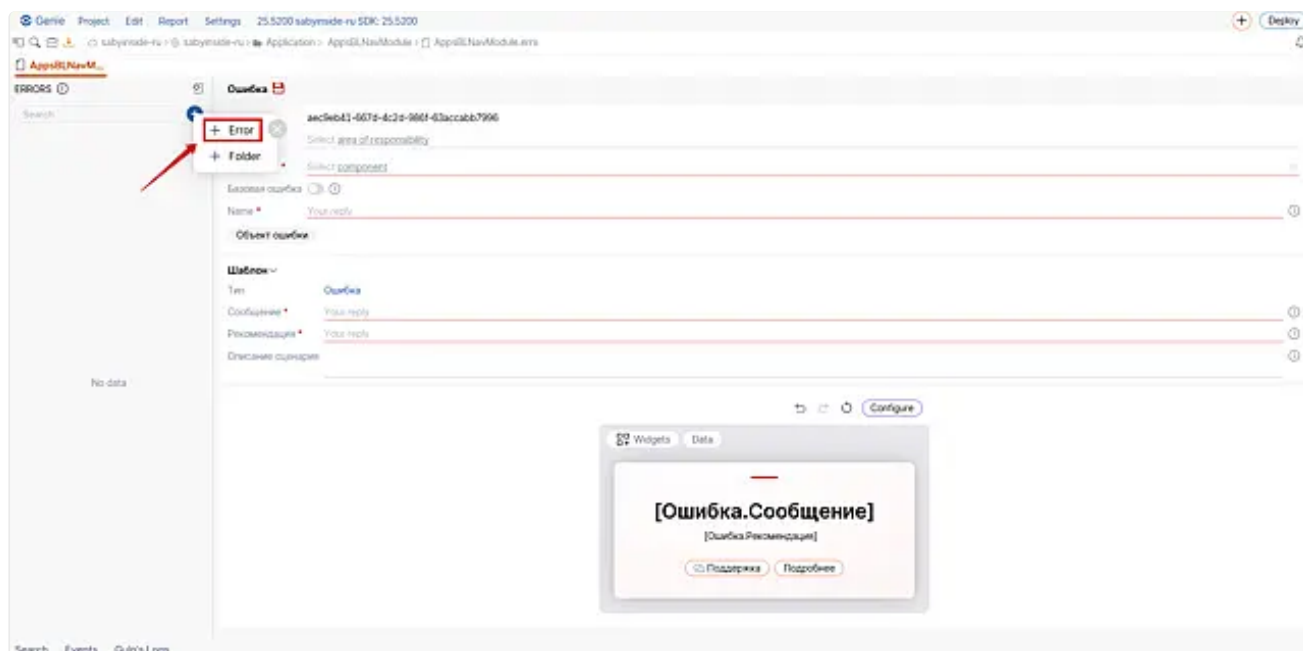
Как создать кастомизируемую ошибку

Создать декларацию в Genie

Создайте errx-файл с помощью контекстного меню модуля.



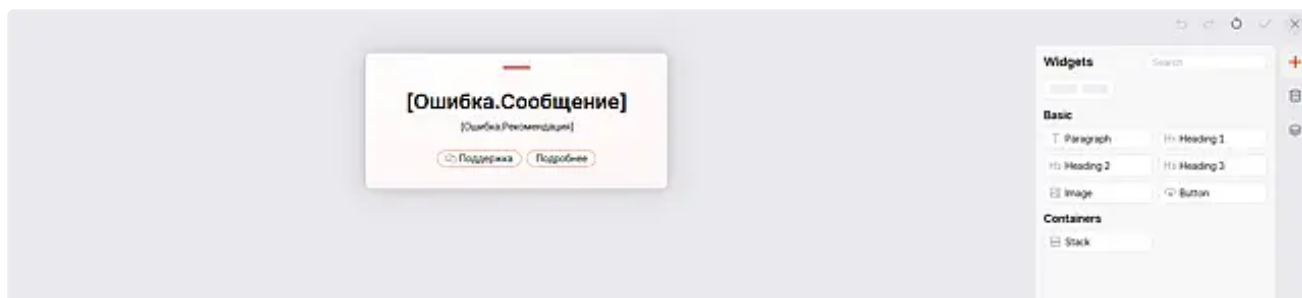
Нажмите "+" и выберите "Error" для создания новой ошибки.



В разделе **Шаблон** заполните поля, с помощью которых задается отображение ошибки для пользователя:

- **Тип** - отвечает за внешний вид фрейма. В зависимости от выбранного типа (Ошибка, Предупреждение или Информация), меняется маркер информационного окна.
- **Сообщение** - короткий текст ошибки для пользователя. Соответствует полю **[Ошибка.Сообщение]**.
- **Рекомендация** - детальное описание ошибки. Соответствует полю **[Ошибка.Рекомендация]**.
- **Описание сценария** - информация о том, что пользователь должен сделать для исправления ошибки.

С помощью конструктора вы можете настроить окно ошибки: добавить виджеты, колонки, параграфы, изображения, вставить поля с данными из контекста и т.п. Для перехода к настройке нажмите кнопку Configure:



Создать класс ошибки в коде

Python

Создаем тип ошибки

```
1 from customizable_error_base import PythonCustomizableException
2
3 class MyCustomException(PythonCustomizableException):
4     StaticId = 'e86f23f0-bd33-46b2-95f6-42e3d6bd9fc3'
5
6     def __init__(self, details, user_msg, error_code=-1,
7 type=sbis.ExceptionType.ERROR,
8         passed_exception=None, additional_data=sbis.Record()):
9         super().__init__(details, user_msg, error_code=error_code,
10 type=type,
11         passed_exception=passed_exception,
12         additional_data=additional_data,
13         class_uuid=MyCustomException.StaticId)
```

1. Тип должен быть наследником базового класса всех кастомизируемых ошибок PythonCustomizableException.
2. Значение поля StaticId должно совпадать с Uuid'ом из Genie.
3. Сигнатура конструктора должна в точности совпадать с примером. Это boilerplate код, необходимый для корректной передачи объектов исключений между языками и сервисами.

Регистрируем тип ошибки в системе

```
1 import sbis
2 from source import MyCustomException
3 from customizable_error_base import RegisterCustomizableError
4
5 def OnEndAllLoadModules(answer):
6     RegisterCustomizableError(MyCustomException)
7     return True
8
```

1. Создаем [обработчик события](#) загрузки сервисного модуля.
2. Вызываем функцию RegisterCustomizableError, в качестве аргумента передаем **тип** ошибки.

Добавляем выброс исключения в код

Для этого модуль customizable_error_base предоставляет две функции:

```
1 def ThrowExceptionError(typoname, rec=Record()):
2     """Выброс кастомизируемого исключения с ошибкой"""
3     pass
```

```

4
5 def ThrowExceptionWarning(typename, rec=Record()):
6     """Выброс кастомизируемого исключения с предупреждением"""
7     pass
8

```

Их различие заключается в политике вывода сообщения в лог.

Пример вызова, который бросит кастомизируемую ошибку и напишет предупреждение в лог сервиса:

```

1 ThrowExceptionWarning(MyCustomException)

```

C++

Создаем тип ошибки

*.hpp

```

1 #include <sbis-customizable-errors/error.hpp>
2
3 namespace sbis::myerror
4 {
5
6 DECLARE_CUSTOM_ERROR_HEAD( MyCustomCppException, "{9d007d98-6896-4438-
7 9edd-f2d23cbcfe61}" );
8 }

```

*.cpp

```

1 DECLARE_CUSTOM_ERROR_IMPL_BODY( MyCustomCppException );

```

Это простейший пример декларации неэкспортируемой кастомизируемой ошибки.

Другие макросы создания ошибок:

```

1 #define DECLARE_CUSTOM_ERROR_HEAD( ClassName, ClassId ) ...
2
3 #define DECLARE_EXPORT_CUSTOM_ERROR_HEAD( ClassName, ClassId, Export )
4     ...
5
6 #define DECLARE_DERIVED_CUSTOM_ERROR_HEAD( ClassName, BaseClassName,
7     ClassId ) ...
8
9 #define DECLARE_EXPORT_DERIVED_CUSTOM_ERROR_HEAD( ClassName,
10     BaseClassName, ClassId, Export ) \

```

Регистрируем тип ошибки в системе

*.cpp

```

1 REGISTER_CUSTOM_ERROR( MyCustomCppException );

```

Этот код создает глобальную статическую переменную. Если ваш модуль поддерживает [статическую сборку](#), то регистрацию необходимо выполнять в отдельной функции.

Добавляем выброс исключения в код

Для этого модуль `customizable_error_base` предоставляет функции:

```
1 namespace sbis::error
2 {
3     template< DerivedFromCustomizableErrorBase Err >
4     [[noreturn]] void ThrowCustomizableError();
5
6     template< DerivedFromCustomizableErrorBase Err >
7     [[noreturn]] void ThrowCustomizableWarning();
8
9     template< DerivedFromCustomizableErrorBase Err >
10    [[noreturn]] void ThrowCustomizableError( Record const& );
11
12    template< DerivedFromCustomizableErrorBase Err >
13    [[noreturn]] void ThrowCustomizableWarning( Record const& );
14 }
```

Их различие заключается в политике вывода сообщения в лог и возможностью прикрепить к ошибке контейнер с данными.

Пример вызова, который бросит кастомизируемую ошибку и напишет ошибку в лог сервиса:

```
1 #include <sbis-customizable-errors/error.hpp>
2
3 error::ThrowCustomizableWarning< MyCustomCppException >();
```

Наследование ошибок

Модуль предоставляет возможность создания базовых типов ошибок и выстраивания дерева наследования. С точки зрения вызывающего кода, перехват ошибок будет работать согласно правилам написания кода. Базовую ошибку невозможно выбросить, но можно перехватить через `try/catch` блок.

Python

Создание базового типа ошибок

```
1 from customizable_error_base import PythonCustomizableException
2
3 class MyPyCustomException(PythonCustomizableException):
4     StaticId = 'e86f23f0-bd33-46b2-95f6-42e3d6bd9fc3'
5
6     def __init__(self, details, user_msg, error_code=-1,
7 type=sbis.ExceptionType.ERROR,
8         passed_exception=None, additional_data=sbis.Record()):
9         super().__init__(details, user_msg, error_code=error_code,
10 type=type,
11         passed_exception=passed_exception,
12         additional_data=additional_data,
13         class_uuid=MyPyCustomException.StaticId)
14
15     @property
16     def own_error_type_signature(self):
17         return super().own_error_type_signature +
18 [sbis.UUID(MyPyCustomException.StaticId)]
```

Необходимо переопределить свойство `own_error_type_signature` аналогично примеру.

Регистрация базового типа ошибки в системе

```
1 from sbis import ExceptionList
2
3 def OnEndAllLoadModules(answer):
4     ExceptionList().register(MyPyCustomException)
5     ExceptionList().register_base_handle(MyPyCustomException.StaticId)
6     return True
```

Регистрация производится так же в событии загрузки модуля, но другим набором функций.

C++

Регистрация базового типа ошибки в системе

```
1 REGISTER_BASE_CUSTOM_ERROR( MyClass );
```

Ошибка объявляется и создается аналогично базовой, но регистрация производится специальным макросом.